OOPS CONCEPTS

What is OOPS?

- OOPS stands for **Object Oriented Programming System**.
- It organizes a program using **objects** instead of functions.
- OOPS makes programs:
    - Easy to understand
    - Easy to maintain
    - Reusable

Class

- A **class** is a **blueprint or template**.
- It defines **data members (variables)** and **member functions**.
- Class does not occupy memory until an object is created.

Syntax:	**Example:**

`class ClassName`
`{`
`    data members;`
`    member functions;`
`};`
	`class Student`
	`{`
	`public:`
	`  int roll;  // data member`
	`  void show()  // member function`
	`    {`
	`        cout << roll;`
	`    }`
	`};`

Object

- An **object** is an **instance of a class**.
- Object represents a **real-world entity**.

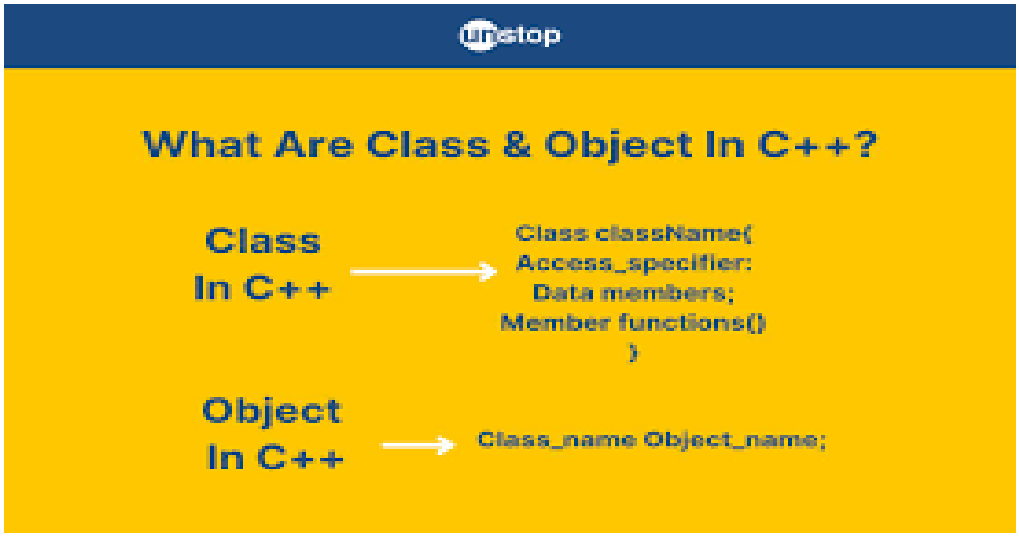
- Object occupies memory.

Syntax:

Class_name object_name;

Example:

```
Student s1;
```



Access Specifiers

Access specifiers determine the visibility of class members:

1. **Public:** Members are accessible from anywhere in the program.
2. **Private:** Members are accessible only within the class itself.
3. **Protected:** Members are accessible within the class and by derived classes.

Constructor

- A **constructor** is a special function.
- It is called **automatically when an object is created**.
- Used to **initialize data members**.
- The constructor name is the same **as the class name**.
- It has **no return type**.

```
#include <iostream.h>
#include <conio.h>

class Test
{
    int a;
public:
    Test()   // Constructor
    {
        a = 10;
        cout << "Constructor called";
    }
};

void main()
{
    clrscr();
    Test t1;   // Constructor called automatically
    getch();
}
```

```
Output:-
Constructor called
```

Destructor

Explanation:

- Destructor name starts with ~
- Called automatically when object is destroyed
- Used to free memory

```
#include <iostream.h>
#include <conio.h>

class Test
{
public:
    ~Test()   // Destructor
```

```
    {
        cout << "Destructor called";
    }
};

void main()
{
    clrscr();
    Test t1;   // Object created
    getch();
}              // Destructor called here
```

```
Output:-
Destructor called
```

Structure of a C++ Program

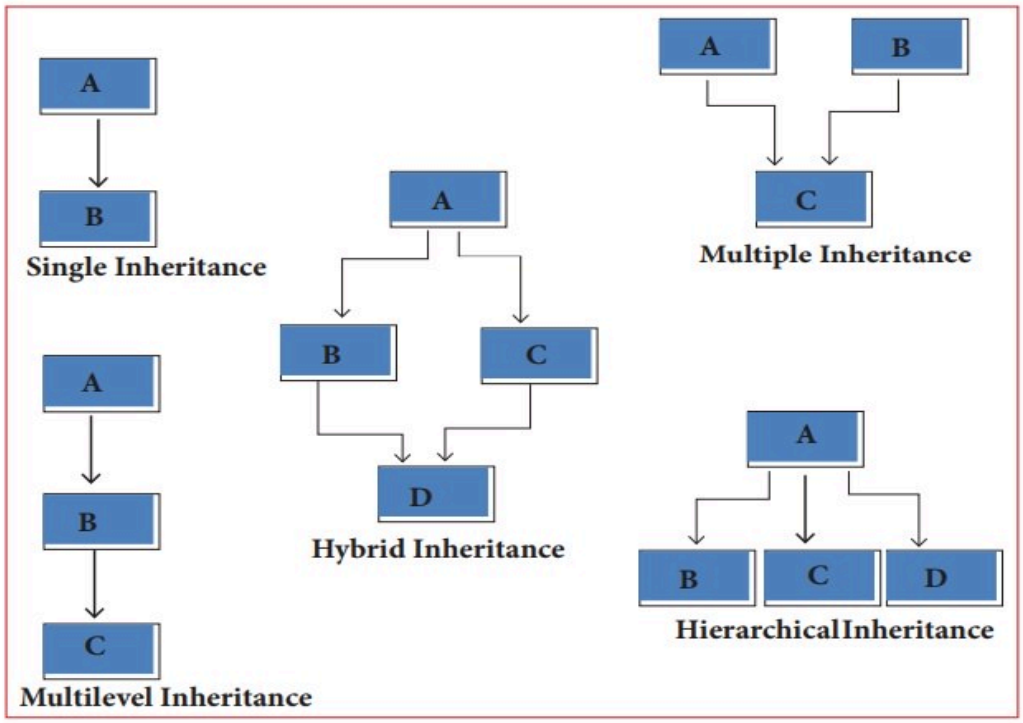
```
#include <iostream> ← Preprocessor Directive (includes input/output library)
using namespace std;    ← Namespace Declaration (avoids writing std::)

int main()              ← Main Function (program execution starts here)
{
  int a = 10;         ← Variable Declaration (stores value 10 in variable a)
  cout << a;          ← Statement / Expression (prints value of a)
  return 0;           ← Return Statement (ends the program)
}
```

```
Output:-
10
```

Inheritance

- Inheritance is a feature of Object-Oriented Programming (OOP) that allows a new class (derived class) to inherit attributes and behaviors from an existing class (base class).
- It promotes code reusability and hierarchical classification.

Types of Inheritance:



1. Single Inheritance - One base class, one derived class.

2. Multiple Inheritance - One derived class inherits from multiple base classes.

3. Multilevel Inheritance - A derived class inherits from another derived class.

4. Hierarchical Inheritance - Multiple derived classes inherit from a single base class.

5. Hybrid Inheritance - Combination of different inheritance types.

Basic Syntax of Inheritance

```
class BaseClass
{
   // members
};
class DerivedClass : access_specifier BaseClass
{
   // additional members
};
```

```
#include <iostream.h>
#include <conio.h>

class A
{
public:
    void show()
    {
        cout << "This is Parent class";
    }
};

class B : public A   // Inheritance
{
};

void main()
{
    clrscr();
    B obj;
    obj.show();   // Accessing parent class function
    getch();
}
```

```
Output:-

This is Parent class
```

```
#include <iostream>
```

```
class Person
{
public:
    string name;
    int age;

    void getData()
    {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Age: ";
        cin >> age;
    }
};

class Student : public Person
{
public:
    int roll_no;

    void showData()
    {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "Roll No: " << roll_no << endl;
    }
};

void main()
{
    Student s;
    s.getData();
    cout << "Enter Roll No: ";
    cin >> s.roll_no;
    s.showData();
    getch();
}
```

Abstract Classes

- Abstraction is the process of hiding internal implementation details and showing only the essential features of an object to the user.
- In simple words, we show what an object does, not how it does it.

Real-Life Example

Think about an ATM Machine
You can:

- Insert card
- Enter PIN
- Withdraw money

But you don't know how the machine processes transactions internally.
That hidden logic is abstraction.

- An **abstract class** contains **at least one pure virtual function**.
- It **cannot be instantiated** (object cannot be created).
- Used to **force derived classes to implement functions**.

Example:

```
#include <iostream.h>

class Shape
{
public:
   virtual void draw() = 0;   // Pure virtual function
};

class Circle : public Shape
{
public:
   void draw()
   {
      cout << "Drawing Circle";
   }
};
```

```
Void main()
{
   Circle c;
   c.draw();
   getch();
}
```

Output:-
Drawing Circle